

SD Basic - Program Structure

Programs, subroutines and functions; variables, matrices, common blocks and how a program is called.

This page covers the shape of an SD BASIC program: how it is declared, how it is called, what its variables are, and the declarations that have to appear before anything else.

SD folds case, so a program may be written in either case. Keywords are shown here in lower case. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

The three kinds of program

```
program {name}  
subroutine name {(argument, ...)} {var.args}  
function name {(argument, ...)} {var.args}
```

program	runs on its own. Started by <code>run</code> , by its catalogued name, or from another program with <code>execute</code>
----------------	--

subroutine	called by <code>call</code> , and returns nothing. Its arguments carry values back
-------------------	--

function	called inside an expression, and returns a value with <code>return</code>
-----------------	---

A source record with no declaration at all is a program. `end` closes it.

```
program hello  
  print 'hello'  
end
```

`var.args` allows the caller to pass fewer arguments than are declared; `arg.count()` then says how many actually arrived, and reading one that was not passed is an unassigned-variable error rather than a silent empty string.

LOCAL SUBROUTINES AND FUNCTIONS

```
local subroutine name {(arguments)}
```

```
local function name {(arguments)}
```

These live inside the program that uses them rather than in the catalogue, and they can declare `private` variables of their own — which is how you get a routine whose working variables do not collide with the rest of the program.

They are not called the way external ones are. A local subroutine is reached with `gosub name(...)` and not `call`; a local function must be declared with `deffun ... local` before it is used; and both need an explicit `return`, because falling into the closing `end` stops the program. All of that, with the scope rules and the errors each mistake produces, is on SD Basic - Modern Program Structure.

INTERNAL SUBROUTINES

```
gosub label
...
label:
...
return
```

`gosub` is not a call to another program — it jumps within the current one, and shares all its variables. The nesting limit is **256 levels**.

Calling

```
call name {(argument, ...)}
call @variable {(argument, ...)}
subr(name {, argument, ...})
```

<code>call name</code>	the name is fixed at compile time
<code>call @variable</code>	the name is taken from the variable at run time
<code>subr()</code>	the function form — use it where you want the result in an expression

Arguments are passed by reference, not by value. A subroutine that assigns to one of its parameters changes the caller's variable. That is how a subroutine returns anything at all, and it is also how a subroutine that uses a parameter as scratch space quietly corrupts its caller's data. **If you do not intend to return a value through an argument, copy it into a local variable first.**

Passing an expression rather than a variable — `call sub(a + 0)` — forces a copy, because there is no variable to write back to.

```
catalogued(name)
```

reports whether a name can be called. `catalogued('ZZNOSUCH')` is `0`. Use it before a `call @variable` whose name came from data, because an uncatalogued name is a runtime abort rather than a testable failure.

Variables

Variables are not declared and have no type. A variable holds whatever was last assigned to it, and its type follows the value.

```
assigned(variable)
unassigned(variable)
vartype(variable)
```

EXPRESSION	RESULT
<code>assigned(neverset)</code>	<code>0</code>
<code>assigned(zz)</code> after <code>zz = 1</code>	<code>1</code>
<code>unassigned(neverset)</code>	<code>1</code>

The compiler warns about a variable that is never assigned — *"WARNING: NEVERSET is not assigned a value"* — but it still compiles, and using the variable is an error only when the line is reached.

```
VARTYPE()
```

Returns the internal type code:

VALUE	CODE
unassigned	<code>0</code>
an integer	<code>2</code>
a floating-point number	<code>3</code>
a string	<code>5</code>

The remaining codes exist for things a program does not usually inspect: `1` address, `4` subroutine reference, `6` file variable, `7` array, `8` common block, `9` image, `11` select list, `13` socket, `15` object.

`vartype()` **tells you how the value is held, not what it means.** `'12'` read from a file is a string (`5`) even though it is a number everywhere it is used; the same value after arithmetic is an integer (`2`). **Use `num()` to ask whether something is numeric** — see SD Basic - Math Functions — and keep `vartype()` for telling an unassigned variable from an empty one.

`set.unassigned` returns a variable to the unassigned state, which is how you make `assigned()` false again after using it.

Constants

```
equate name to value
equ name to value
```

`equate` is resolved at compile time and costs nothing at run time. It can name a literal, another variable, or a matrix element:

```
equate true to 1
equate esc to char(27)
equate cust.name to record<3>
```

An `equate` **is a substitution, not a variable.** You cannot assign to one, and it is replaced wherever the name appears.

`equate cust.name to record<3>` means `cust.name` reads field 3 of whatever `record` holds at that moment.

```
$include file.name record.name
include file.name record.name
```

pulls in another source record at compile time. The system definitions live in `syscom` — `$include syscom keys.h` for the key numbers used by `fileinfo()`, `selectinfo()` and the rest.

Matrices

```
dim name(rows {, columns})
dimension name(rows {, columns})
mat name = expression
mat name = mat other.name
```

A matrix is a fixed set of numbered variables, not a dynamic array. `dim` must appear before any use, and the bounds are fixed at that point.

```
dim m(3)
mat m = 7          ;* every element becomes 7
```

After `mat m = 7`, `m(1)`, `m(2)` and `m(3)` all read `7`.

A matrix has a zero element and it is not counted in the dimension. `dim m(3)` gives `m(0)` through `m(3)`. `m(0)` is where `matparse` puts anything left over, and it is easy to forget it exists.

BETWEEN A MATRIX AND A STRING

```
matbuild variable from matrix {, start {, end}}
matparse matrix from string, delimiter
inmat({matrix})
```

<code>matbuild</code>	joins the elements into one string
<code>matparse</code>	splits a string across the elements
<code>inmat()</code>	after <code>matparse</code> , how many elements were filled

With `m` holding `7`, `7`, `7`:

<code>matbuild bs from m</code>	the three elements joined with field marks
<code>matbuild bs from m, 1, 2</code>	the first two only
<code>matparse m from 'x' : @vm : 'y' : @vm : 'z', @vm</code> then <code>m(1) ... m(3)</code>	<code>x y z</code>
<code>inmat()</code> after that	<code>3</code>

The `using` clause of `matbuild` does not work, in either case. The documented syntax `matbuild var from mat using delimiter` compiles the keyword as a **variable name** and the program aborts at run time with *"Unassigned variable USING"*, with both `using` and `USING`.

This is inherited, not introduced by this port — the `st.matbuild` block in `BCOMP` is byte-identical to the one in the upstream `sdb64` tree. **The delimiter is always a field mark.** If you need another one, use `matbuild` and then `change()`, or build the string with a `for` loop.

`matparse` splits on the delimiter you give and stops when the matrix is full; anything left over goes into element zero.

Common blocks

```
common {/name/} variable, ...
clearcommon
delete.common name
```

Variables in a common block survive `chain` and are shared between programs that declare the same block. An **unnamed** common block — `common // a, b, c` — belongs to the program run and is cleared when it ends; a **named** one persists for the session.

`//` is the unnamed common marker, which is why it is not an integer-divide operator. See SD Basic - Math Functions.

`clearcommon` sets every unnamed common variable to zero. `delete.common` discards a named block entirely, which is the only way to reclaim it within a session.

A common block is matched by name and position, not by variable name. Two programs declaring the same block with different variable lists will read each other's data at the wrong offsets, silently. **Put the declaration in an include record and `$include` it everywhere**, so there is one copy.

Class modules and objects

SD BASIC has class modules with member variables, methods, properties, constructors, destructors and inheritance. **They have their own page**, because they are a different way of writing a program rather than a feature of this one: SD Basic - Modern Program Structure.

The short form:

```
class name {inherits base}
private variable, ...      public variable, ...
public subroutine name(...) public function name(...)
get name                  set name(value)
end
```

```
o = object('CLASSNAME')
o->method(args)
```

```
o->property = value
```

SD's own client library is a class module. `gp1.bp/SDCLIENT` is 1,040 lines with 33 members and both lifecycle hooks — connecting to another SD server from BASIC means instantiating it. *(An earlier version of this page said nothing in SD used class modules. That was wrong.)*

Arguments

```
arg.count()  
arg(n)  
set.arg n, value
```

These read the arguments of the **command line** that started the program, not the arguments of a subroutine call. `arg(1)` is the first word after the verb.

Discarding a result

```
void expression
```

Calls a function and throws the answer away. It exists because SD BASIC has no statement form for a function call — `void kernel(...)` is the idiom you will see throughout `gp1.bp`.

What is not here

Nothing in the program-structure group has been removed from this port. The `package` statements — `enter.package`, `exit.package` and `package` — went before SD Core, and `varset` is present but is an internal debugging aid rather than something application code uses.

See also

SD Basic - Program Control · SD Basic - Dynamic Arrays · SD Basic - File Handling · SD Basic - Math Functions.
